

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)
Кафедра экономической математики, информатики и статистики (ЭМИС)

РЕКУРСИВНЫЕ ФУНКЦИИ

Отчёт по лабораторной работе по дисциплине «Программирование»

Студент гр.594-2

 А. А. Танцеров

“ 26 ” Февраля 2025 г.

Руководитель канд. физ.- мат. наук,
доцент каф. ЭМИС ТУСУР

_____ Е. А. Шельмина

“ _____ ” _____ 2025 г.

Томск 2025

Лабораторная работа №7

Рекурсивные функции

Цель работы: получение навыков в написании программ с использованием рекурсивных функций.

Рекурсивной (самовызываемой или самовызывающей) называют функцию, которая прямо или косвенно вызывает сама себя.

При каждом обращении к рекурсивной функции создается новый набор объектов автоматической памяти, локализованных в коде функции.

Возможность прямого или косвенного вызова позволяет различать прямую или косвенную рекурсии. Функция называется косвенно рекурсивной в том случае, если она содержит обращение к другой функции, содержащей прямой или косвенный вызов первой функции. В этом случае по тексту определения функции ее рекурсивность (косвенная) может быть не видна. Если в функции используется вызов этой же функции, то имеет место прямая рекурсия, т.е. функция по определению рекурсивная.

В рекурсивных функциях необходимо выполнять следующие правила:

1. при каждом вызове в функцию передавать данные;
2. на каком-то шаге должен быть прекращен дальнейший вызов этой функции, это значит, что рекурсивный процесс должен шаг за шагом упрощать задачу так, чтобы для нее появилось нерекурсивное решение, иначе функция будет вызывать себя бесконечно;
3. после завершения очередного обращения к рекурсивной функции в вызывающую функцию должен возвращаться некоторый результат для дальнейшего его использования.

И нередко циклические конструкции более эффективны, чем рекурсия. Например, если функция вызывает себя тысячи раз, потребуются большой объем стековой памяти для хранения копий значений аргументов и адреса возврата для каждого вызова, что может привести к тому, что программа быстро исчерпает выделенную для нее память стека, поскольку объем памяти

стека обычно фиксирован и ограничен. что может привести к аварийному завершению программы. Поэтому в таких случаях, как правило, лучше использовать альтернативные подходы, например цикл. Однако, несмотря на накладные расходы, использование рекурсии часто может значительно упростить написание программы.

Задание 1. Составить рекурсивную функцию для решения задачи согласно варианту, представленному на рисунке 1.

11	Составить рекурсивную функцию <code>double PowerN(double x, int n)</code> вещественного типа, находящую значение n -й степени числа x по формуле: $x^0 = 1$, $x^n = x \cdot x^{n-1}$ при $n > 0$, $x^n = 1 / x^{-n}$ при $n < 0$.
----	--

Рисунок 1 – Скриншот варианта к заданию 1

Для выполнения задания 1 создаём функцию `double PowerN`, в которую входят две переменные, рассматриваем три случая, представленные в формулировке задания, вызывая рекурсивные функции, где это необходимо. В `main` вызываем русификатор текста, создаём переменные, присваиваем значения, затем выводим результат, вызывая функцию. Результат работы представлен на рисунке 2.

```

1  #include <iostream>
2  using namespace std;
3
4  double PowerN(double x, int n) {
5      if (n == 0) return 1;
6      if (n > 0) return x * PowerN(x, n - 1);
7      if (n < 0) return (1 / PowerN(x, -n));
8  }
9
10 int main() {
11     setlocale(LC_ALL, "RU");
12     double x; int n;
13     cout << "Введите число x -> "; cin >> x;
14     cout << "Введите степень n -> "; cin >> n;
15     cout << "Результатом работы является -> " << PowerN(x, n);
16 }
  
```

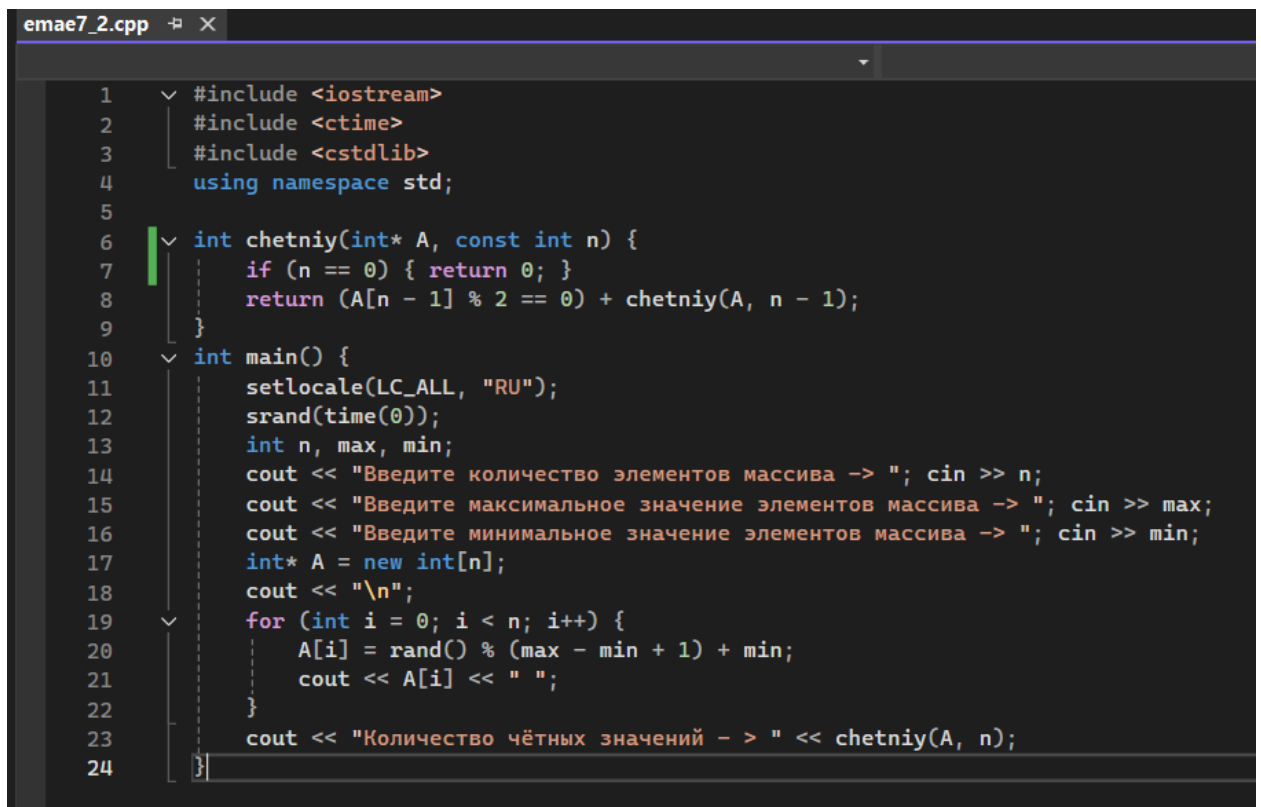
Рисунок 2 – Скриншот кода к заданию 1

Задание 2. Составить рекурсивную функцию для решения задачи согласно варианту, представленному на рисунке 3.

№ варианта	Задание
1	В массиве целых чисел с помощью рекурсивной функции найти количество четных элементов.

Рисунок 3 – Скриншот варианта к заданию 2

Вызываем две дополнительные библиотеки для генерации произвольных чисел. Создаём функцию `int chetnyy`, в которую передаём два значения, возвращать данная функция количество чётных чисел, проверяя, чётное ли данное число, затем вызывая на элемент меньше, пока не дойдёт до конца. Аналогично создаём и присваиваем переменные, с помощью указателя создаём массив из `n` элементов. С помощью цикла `for` заполняем массив произвольными значениями. Вызываем функцию, получаем ответ. Результат работы представлен на рисунке 4.



```
емае7_2.cpp
1  #include <iostream>
2  #include <ctime>
3  #include <cstdlib>
4  using namespace std;
5
6  int chetnyy(int* A, const int n) {
7      if (n == 0) { return 0; }
8      return (A[n - 1] % 2 == 0) + chetnyy(A, n - 1);
9  }
10 int main() {
11     setlocale(LC_ALL, "RU");
12     srand(time(0));
13     int n, max, min;
14     cout << "Введите количество элементов массива -> "; cin >> n;
15     cout << "Введите максимальное значение элементов массива -> "; cin >> max;
16     cout << "Введите минимальное значение элементов массива -> "; cin >> min;
17     int* A = new int[n];
18     cout << "\n";
19     for (int i = 0; i < n; i++) {
20         A[i] = rand() % (max - min + 1) + min;
21         cout << A[i] << " ";
22     }
23     cout << "\nКоличество чётных значений -> " << chetnyy(A, n);
24 }
```

Рисунок 4 – Скриншот кода к заданию 2

Вывод: получены навыки в написании программ с использованием рекурсивных функций.